

Stacks and Queues

Stacks and Queues are linear data structures that restrict how elements are added and removed. They are the fundamental building blocks for many algorithms.

A Stack follows Last-In-First-Out (LIFO) order. Think of a stack of plates: you add (push) to the top, and you remove (pop) from the top.



Operations:

- push(x): Add x to the top.
- pop(): Remove and return the top element.

A Queue follows First-In-First-Out (FIFO) order. Think of a line at a store: you join at the back (enqueue), and you are served from the front (dequeue).



Operations:

- enqueue(x): Add x to the back.
- dequeue(): Remove and return the front element.

Let's compare them side-by-side.

Feat	ts	Stack	Queue
Op	del	LIFO	FIFO
Insert	at head	at tail	at tail
Remove	at head	at head	at tail

Let's trace a sequence of stack operations to see LIFO in action.

Start empty. push(5), push(3), push(8), pop(), push(2).

Step 1: push(5) → [5]

Step 2: push(3) → [5, 3]

Step 3: push(8) → [5, 3, 8]

Step 4: pop() → returns 8. Stack is now [5, 3]

Step 5: push(2) → [5, 3, 2]

Notice how pop() removed 8, the most recently added element!

Where do we use these?

- Stack: Function call management (call stack), Undo/Redo mechanisms, Expression evaluation (parsing parentheses).
- Queue: Breadth-First Search (BFS) in graphs, CPU task scheduling, Printer job queues.

How do we build them?

- Array-based: Use a dynamic array. For queues, we use a circular array to avoid $O(n)$ shifting when dequeuing!
- Linked-list-based: Use nodes with pointers. Stack: insert/delete at head. Queue: insert at tail, delete at head.

Operation	Stack	Queue
Insert	$O(1)$	$O(1)$
Remove	$O(1)$	$O(1)$
Peek	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$

Stack = LIFO (plate stack), Queue = FIFO (line)